Dynamic Memory Allocation Functions in C

1 Introduction

Dynamic Memory Allocation (DMA) allows memory to be allocated at runtime using functions from <stdlib.h>.

2 malloc()

Definition: malloc() (Memory Allocation) allocates a specified number of bytes and returns a pointer to the allocated memory.

Syntax:

ptr = (type*) malloc(size_in_bytes);

Key Point: Memory is **not initialized** (contains garbage values).

Example:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*) malloc(3 * sizeof(int));

    ptr[0] = 10;
    ptr[1] = 20;
    ptr[2] = 30;

    printf("%d %d %d", ptr[0], ptr[1], ptr[2]);

    free(ptr);
    return 0;
}
```

3 calloc()

Definition: calloc() (Contiguous Allocation) allocates memory for multiple elements and initializes them to zero.

Syntax:

ptr = (type*) calloc(number_of_elements, size_of_each);

Key Point: Memory is **initialized to 0**.

Example:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*) calloc(3, sizeof(int));

    printf("%d %d %d", ptr[0], ptr[1], ptr[2]);

    free(ptr);
    return 0;
}
```

4 realloc()

Definition: realloc() resizes previously allocated memory.

Syntax:

ptr = realloc(ptr, new_size);

Key Point: Used to increase or decrease memory size.

Example:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr;
    ptr = (int*) malloc(2 * sizeof(int));

    ptr[0] = 5;
    ptr[1] = 10;

    ptr = (int*) realloc(ptr, 4 * sizeof(int));

    ptr[2] = 15;
    ptr[3] = 20;

    printf("%d %d %d %d", ptr[0], ptr[1], ptr[2], ptr[3]);

```

```
    free(ptr);
    return 0;
}
```

5 free()

Definition: free() releases dynamically allocated memory back to the system.

Syntax:

free(ptr);

Important: Always free memory to avoid **memory leaks**.

Example:

```
int *ptr = (int*) malloc(sizeof(int));
*ptr = 100;

free(ptr);
ptr = NULL;      // Good practice
```

6 Conclusion

- malloc() – Allocates memory (not initialized)
- calloc() – Allocates and initializes to zero
- realloc() – Resizes allocated memory
- free() – Deallocates memory

Program 1: Netflix Movie Using `malloc()`

(Example inspired by Netflix)

Features:

- Allocate memory dynamically
- Input movie details
- Print movies released after 2010
- Find max and min rating

C Program Using `malloc()`

```
#include <stdio.h>
#include <stdlib.h>

struct Movie {
    char title[50];
    int releaseYear;
```

```
    float rating;
};

int main() {
    int n, i;

    printf("Enter number of movies: ");
    scanf("%d", &n);

    // Dynamic memory allocation
    struct Movie *m = (struct Movie*) malloc(n * sizeof(struct
Movie));

    if(m == NULL) {
        printf("Memory not allocated!\n");
        return 1;
    }

    // Input
    for(i = 0; i < n; i++) {
        printf("\nEnter details for movie %d\n", i+1);

        printf("Title: ");
        scanf(" %[^\n]", m[i].title);

        printf("Release Year: ");
        scanf("%d", &m[i].releaseYear);

        printf("Rating: ");
        scanf("%f", &m[i].rating);
    }

    float maxRating = m[0].rating;
    float minRating = m[0].rating;
    int maxIndex = 0, minIndex = 0;

    printf("\nMovies released after 2010:\n");

    for(i = 0; i < n; i++) {

        if(m[i].releaseYear > 2010) {
            printf("%s (%d) - %.1f\n",
                   m[i].title,
                   m[i].releaseYear,
                   m[i].rating);
        }

        if(m[i].rating > maxRating) {
            maxRating = m[i].rating;
            maxIndex = i;
        }

        if(m[i].rating < minRating) {
```

```
            minRating = m[i].rating;
            minIndex = i;
        }
    }

    printf("\nMovie with Maximum Rating:\n");
    printf("%s - %.1f\n", m[maxIndex].title,
m[maxIndex].rating);

    printf("\nMovie with Minimum Rating:\n");
    printf("%s - %.1f\n", m[minIndex].title,
m[minIndex].rating);

    // Free allocated memory
    free(m);

    return 0;
}
```